SAMSUNG SW AI ACADEMY FOR YOUTH

對外秘

프로젝트 명세서

ChatBot Project

(Java Track)

“자율 도메인”

4 회차

對外秘

목차

1. ChatBot Project 명세서 3

1.1 목표 3

1.2 준비사항 3

1.3 작업 순서 5

1.4 요구사항 5

1.5 참고자료 8

1.6 프로젝트 결과 9

1. ChatBot Project 명세서

1.1 목표

이번 'ChatBot Project 명세서' 프로젝트의 목표는 다음과 같습니다.

- 생성형 AI 의 기본 동작 방식을 이해하기 위해 외부 API 호출을 통한 ChatBot 기능을 구현한다. (본 프로젝트에서는 ChatGPT 를 예제로 사용)
- HTML, CSS, JavaScript 를 활용하여 사용자 입력, 버튼 클릭, 응답 출력 등 기본적인 사용자 인터랙션이 적용된 웹 화면을 구성한다.
- JavaScript 의 fetch API 를 이용하여 클라이언트에서 서버로 요청을 보내고 응답을 처리하는 전체 흐름을 경험한다.
- FastAPI 로 구현된 AI ChatBot 서버를 제공받아 이를 하나의 외부 서비스(API)로 활용하는 동시에, 서버 내부 코드의 구조를 단계적으로 이해하고 개선 방향을 고민한다.
- 문서 검색 기반 응답 방식인 RAG(Retrieval-Augmented Generation)의 개념을 이해하고, 관심 있는 주제의 문서나 데이터를 직접 수집하여 파일 업로드 방식으로 RAG 에 적용해본다.
- 단순히 기능 구현에 그치지 않고, RAG 방식의 장점과 한계, 검색 품질, 응답 신뢰성에 대해 고찰 보고서를 작성함으로써 생성형 AI 활용에 대한 비판적 사고를 기른다.

1.2 준비사항

1) 프로젝트 구조

- 본 프로젝트는 프론트엔드와 서버 영역이 분리된 구조로 구성된다. 교육생은 HTML, CSS, JavaScript 를 중심으로 프로젝트를 수행하며, 서버 영역은 제공된 코드를 기반으로 이해 및 확장, 고찰 중심 학습을 진행한다.
  - ■ Front-End 영역
    - ✓ index.html
      - ● 사용자 화면을 구성하는 메인 HTML 파일
      - ● 질문 입력창, 버튼, 응답 출력 영역 등을 포함한다.
    - ✓ style.css
      - ● ChatBot 화면의 레이아웃 및 스타일을 정의한다.
      - ● 메시지 영역 구분, 버튼 스타일, 사용자 인터랙션 시 시각적 효과 등을 담당한다.
    - ✓ script.js
      - ● JavaScript 를 이용하여 사용자 입력을 처리한다.
      - ● fetch API 를 사용해 서버와 통신한다.
      - ● 서버로부터 받은 AI 응답을 화면에 출력하고, 사용자 인터랙션을 제어한다.

- 단계별로 파일을 만들어 활용할 수 있다. (1 단계 기본연결, 2 단계 파일 업로드, 등)

■ Server 영역

✓ 서버 영역은 FastAPI 기반으로 미리 구현된 코드가 제공된다.

✓ 교육생은 서버를 직접 처음부터 구현하지 않으며, 제공된 서버를 분석하고 일부 수정 또는 확장하며 동작을 이해하는 것을 목표로 한다.

✓ main.py
- FastAPI 기반의 AI ChatBot 서버 핵심 코드
- AI API 를 호출하는 기본 기능 제공
- 문서를 임베딩하여 벡터로 저장하는 기능
- 벡터 유사도 기반 검색 기능
- 검색 결과를 AI 응답에 활용하는 RAG 구조
- 파일 업로드 처리 API

✓ data 폴더
- 업로드된 문서 파일이 저장되는 공간
- 사용자가 업로드한 txt, pdf, md 파일이 이 디렉토리에 저장된다.
- 서버 시작 시 또는 업로드 시 해당 문서를 읽어 임베딩 대상으로 사용한다.
- SSAFY Guide.md(교육생 생활 규정 가이드)를 기본 파일로 제공

✓ chroma_data 폴더
- 문서 청크를 임베딩한 벡터 정보가 저장된다.
- 벡터 유사도 검색 시 해당 디렉토리의 데이터를 활용한다.

✓ requirements.txt
- 서버 실행에 필요한 Python 라이브러리 목록
- FastAPI, OpenAI SDK, ChromaDB 등이 포함된다.
- 서버 환경 구성 시 해당 파일을 통해 필요한 패키지를 설치한다.

2) 사용 데이터

- 본 프로젝트에서는 사전에 정해진 데이터셋을 제공하지 않는다.
- 교육생이 관심이 있는 주제에 대한 문서나 자료를 직접 수집하여 활용한다.
- 수집한 데이터의 형식(.md, .txt, .pdf 등)으로 수집하여 활용한다.
- SSAFY Guide.md(교육생 생활 규정 가이드)를 기본 파일로 제공

3) 개발언어 및 툴

- HTML5
- CSS3
- JavaScript (ES6+)
- Python
- Visual Studio Code
- Live Server 확장 프로그램

4) 필수 라이브러리/오픈소스

- OpenAI API(예제)
- FastAPI
- SentenceTransformers

- ChromaDB
- NumPy
- PyPDF

1.3 작업 순서

1) 제공된 프로젝트 구조를 확인하고, 프론트엔드와 서버 영역이 분리되어 있음을 이해한다.
2) Servers 디렉토리 내 requirements.txt 를 통해 서버 실행에 필요한 라이브러리를 설치하고, main.py 의 FastAPI 서버 코드를 실행한다.
3) HTML, JavaScript 를 이용하여 사용자 입력을 서버의 /chat API 로 전달하고 응답을 받아오는 코드를 작성한다.
4) 사용자 인터랙션이 적용된 채팅 화면을 구성한다.
5) 문서 업로드 기능을 구현한다.
6) "/search" API 를 통해 벡터 기반 문서 검색 기능을 구현한다.
7) RAG 기반 통합 ChatBot 기능을 구현 및 확인한다.
8) RAG 서버와 프로젝트 수행 결과를 바탕으로 고찰 및 개선 방향을 정리한다.
9) 프로젝트 결과물을 제출한다.

1.4 요구사항

본 프로젝트는 생성형 AI 를 활용한 완성도 높은 서비스 구현을 목표로 하지 않는다. 대신 웹 서비스 환경에서 생성형 AI 가 어떤 방식으로 호출되고 활용되는지, 그리고 데이터가 AI 응답에 어떤 영향을 미치는지를 이해하는데 목적이 있다.

■ **요구사항 예시 (참고용)**

- 아래의 내용을 참고하여 추가적인 아이디어에 대해 요구사항을 추가 또는 수정하여 기능을 구현합니다. 단, 필수 기능은 반드시 구현해야 하며 수정할 수 없습니다.

기능적 요구사항

번호	분류	요구사항명	요구사항 상세	우선순위
F401	ChatBot	생성형 AI ChatBot 기능	생성형 AI API를 호출하여 사용자 질문에 대한 AI 응답을 제공한다.	필수
F402	ChatBot	사용자 인터랙션 UI	HTML, CSS, JavaScript를 이용해 채팅 형태의 사용자 인터페이스를 구성한다.	필수
F403	ChatBot	API 요청 및 응답 처리	JavaScript fetch API를 이용해 서버 API와 통신한다.	필수
F404	ChatBot	문서 업로드 기능	사용자가 문서를 업로드하여 AI가 참고할 데이터를 제공한다.	심화
F405	ChatBot	벡터 기반 문서 검색	업로드된 문서를 벡터화하여 질문과의 유사도를 검색한다.	심화
F406	ChatBot	RAG 기반 ChatBot 응답	검색된 문서를 활용하여 근거 기반 AI 응답을 생성한다.	심화
F407	ChatBot	고찰 보고서 작성	RAG 구조 및 결과에 대한 분석과 개선 방향을 문서로 정리한다.	심화

1. 기본 기능 (필수)

1) AI ChatBot 기능

① 요구사항 번호: F401

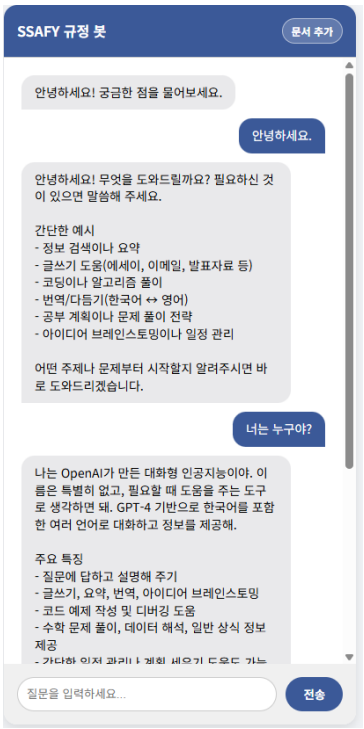
- ◆ 사용자는 웹 화면을 통해 질문을 입력할 수 있어야 한다.
- ◆ 입력된 질문은 JavaScript 를 통해 서버의 /chat API 로 전달되어야 한다.
- ◆ 서버는 외부 생성형 AI API(ChatGPT 등)를 호출하여 응답을 생성한다.
- ◆ 생성된 AI 응답은 JSON 형태로 반환되며, 프론트엔드에서 이를 화면에 출력해야 한다.
- ◆ 네트워크 오류 또는 서버 오류 발생 시 사용자에게 오류 메시지를 안내해야 한다.

② 요구사항 번호: F402

- ◆ HTML 을 이용하여 입력창과 버튼을 구성한다.
- ◆ CSS 를 이용하여 사용자 메시지와 AI 응답을 시각적으로 구분한다.

◆ JavaScript 를 이용하여 버튼 클릭, 입력 이벤트를 처리한다.

◆ 사용자 질문과 AI 응답이 순차적으로 화면에 출력되도록 구현한다.



<그림 1> (예시)ChatBot 화면

③ 요구사항 번호: F403

◆ JavaScript 의 fetch API 를 사용하여 HTTP 요청을 전송한다.

◆ 요청 및 응답 데이터는 JSON 형식을 사용한다.

◆ 비동기 처리 (Promise, async/await)를 활용하여 응답을 처리한다.

◆ 서버로부터 받은 응답을 파싱하여 화면에 출력한다.

2. 심화 기능

1) AI ChatBot 확장 기능

① 요구사항 번호: F404

◆ 사용자는 txt, md, pdf 형식의 문서를 업로드할 수 있어야 한다.

◆ 업로드된 파일은 서버의 data 디렉토리에 저장된다.

◆ 업로드된 문서는 이후 RAG 처리 대상 데이터로 사용된다.

② 요구사항 번호: F405

◆ 문서 텍스트는 일정 길이로 분할(청킹)된다.

◆ 각 청크는 임베딩 벡터로 변환되어 벡터 데이터베이스에 저장된다.

◆ 사용자의 질문 또한 벡터로 변환된다.

◆ 질문 벡터와 문서 벡터 간 유사도 기반 검색을 수행한다.

③ 요구사항 번호: F406

◆ 유사도 기준을 만족하는 문서 청크만 AI 입력으로 활용한다.

◆ 검색 결과가 존재하는 경우, 해당 문서를 근거로 AI 응답을 생성한다.

◆ 검색 결과가 없는 경우, 일반 생성형 AI 응답임을 사용자에게 명시한다.

◆ AI 응답에는 문서 참고 여부를 포함해야 한다.

④ 요구사항 번호: F407

◆ 업로드한 문서에 따라 AI 응답이 어떻게 달라졌는지 분석한다.

◆ 유사도 기준 및 문서 분할 방식이 결과에 미친 영향을 정리한다.

◆ RAG 방식의 장점과 한계를 기술한다.

◆ 개선이 가능하다고 생각되는 부분을 자유롭게 서술한다.

1.5 참고자료

- 공공 데이터 포털
  https://www.data.go.kr/
- HTML, CSS, JavaScript 기능 설명 문서
  https://developer.mozilla.org/ko/
- FastAPI 공식 문서
  https://fastapi.tiangolo.com/ko/
- OpenAI 공식 문서
  https://platform.openai.com/docs/overview
- SSAFY AI 관통 프로젝트 설계 가이드

1.6 프로젝트 결과

본 프로젝트의 최종 산출물은 아래 항목을 포함하며, GIT 의 README.md 파일 또는 Word, PowerPoint 문서를 활용하여 정리한 후 lab.ssafy.com 에 업로드하여 제출한다.

■ **산출물과 제출**

1) 구현 소스코드

◆ HTML, CSS, JavaScript 기반 프론트엔드 전체 소스코드

◆ 서버 프로젝트 전체 소스코드

2) 실행 화면 캡처 자료

◆ ChatBot 실행 화면 캡처(질의, 응답 결과 등)

3) 고찰 보고서(PDF 또는 문서 파일)

◆ 생성형 AI API 호출 흐름에 대한 이해

◆ 문서 업로드 전/후 AI 응답의 차이

◆ RAG 적용 시 장점과 한계

◆ 데이터(문서)의 품질이 응답에 미치는 영향

◆ 향후 개선 또는 확장 가능성 등

4) README.md

◆ 프로젝트 실행 방법

◆ 폴더 구조, 주요 기능 등 내용 작성 필요

5) 제출 방법

◆ ChatBot_[주제명]_지역_반_성명 1_성명 2.zip 파일로 제출

◆ 산출물을 빠짐없이 제출해야 함

- 以上